MSA-DATB09-03 · NEURAL NETWORKS & DEEP LEARNING FOUNDATIONS

Session 4 / 8

PyTorch in depth

The complete training pipeline, every line explained: tensors, autograd, modules, data, loss, optimiser, loop.

Guillaume Saint-Cirgue Albert School · MSc 1 October 2026

Guillaume Saint-Cirgue your instructor

Data Scientist and ML Engineer since 2016

2013 to 2016	Engineering degree, **ENSMM** (Besançon)
2016 to 2017	Data Scientist, **Rolls-Royce**
2017 to 2019	Lead ML Engineer, **GKN Aerospace**
2019	Launched **Machine Learnia** on YouTube: 200,000+ subscribers today
2020	Founded my company and my startup: **30+ AI projects** delivered since
2023	Head of Data, **Airbus**
Today	**ML-PRO** training program: 800 students

To go further

MY YOUTUBE CHANNEL

Machine Learnia

200,000+ subscribers. Machine learning and deep learning videos, including a neural network coded from scratch in NumPy.

youtube.com/@MachineLearnia

MY EBOOK

Apprendre le Machine Learning en une semaine

In English: Learn Machine Learning in One Week.

machinelearnia.com/apprendre-le-machine-learning-en-une-semaine

Both resources are in French. Click the cards in your copy of the course to open them.

PyTorch in six building blocks

In Session 3 you trained `nn.Sequential` with five lines that looked like magic. Today we open each of them.

Block	What it is	In the S03 loop
Tensor	an n-dimensional array, on CPU or GPU	`X`, `y`, every weight
Autograd	records operations, computes gradients by the chain rule	`loss.backward()`
nn.Module	a model: layers declared in `__init__`, used in `forward`	`model(X)`
Loss	a function (prediction, label) to number	`loss_fn(logits, y)`
Optimiser	applies the update rule to every parameter	`optimizer.step()`
DataLoader	serves the data in shuffled mini-batches	new today

Today's data: scikit-learn digits, 1797 images of 8×8 pixels, 10 classes. Small, so everything runs in seconds.

Tensors: shape, dtype, device

```python
import torch
X = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])
print(X.shape, X.dtype, X.device)
# torch.Size([2, 3]) torch.float32 cpu
print(torch.zeros(2, 3).shape, torch.arange(4))
# torch.Size([2, 3]) tensor([0, 1, 2, 3])
print(X @ X.T)       # (2, 3) @ (3, 2) → (2, 2)
# tensor([[14., 32.],
#         [32., 77.]])
print(X.sum(dim=0), X.mean().item())
# tensor([5., 7., 9.]) 3.5
```

- `shape` : size of each dimension. Most bugs are shape bugs: print it.
- `dtype` : `float32` for data and weights, `int64` for class labels.
- `device` : `cpu` or `cuda` . All tensors of one operation must be on the same device.
- `@` is the matrix product, `* + -` are element-wise.
- `dim=0` : reduce over the rows (one result per column).
- `.item()` : a one-element tensor to a Python number.

Broadcasting and the NumPy bridge

```
b = torch.tensor([10., 20., 30.])     # shape (3,)
print(X + b)                          # (2, 3) + (3,)
# tensor([[11., 22., 33.],
#         [14., 25., 36.]])
import numpy as np
a = np.array([1.5, 2.5])              # NumPy float64
t = torch.from_numpy(a).float()       # tensor float32
print(t, t.numpy())
# tensor([1.5000, 2.5000]) [1.5 2.5]
```

Broadcasting rule

Compare the shapes from the right. Two sizes are compatible if they are equal or one of them is 1 (or missing): it is then stretched.

$(2, 3) + (3,)$: **b** is added to every row. $(2, 3) + (2, 1)$: added to every column. $(2, 3) + (2,)$: error.

This is exactly the `+ b` of a layer: `X @ W.T + b` adds the bias vector to every example.

Autograd: the chain rule, automated

```
x = torch.tensor(2.0, requires_grad=True)
f = x**2 + 3*x          # recorded in a graph
print(f)
# tensor(10., grad_fn=<AddBackward0>)
f.backward()            # chain rule, backwards
print(x.grad)           # df/dx = 2x + 3 = 7
# tensor(7.)
```

- `requires_grad=True`: "I will want the gradient with respect to this tensor". Parameters of `nn` layers have it by default.
- Every result keeps a `grad_fn`: the node of the computation graph that knows its local derivative.
- `f.backward()` (on a scalar) walks the graph backwards, multiplies the local derivatives (Session 3) and stores $\partial f / \partial x$ in `x.grad`.

Autograd: accumulation, no_grad, detach

```
w = torch.tensor(1.0, requires_grad=True)
for _ in range(2):
    loss = (3 * w) ** 2      # dloss/dw = 18w = 18
    loss.backward()
print(w.grad)                # tensor(36.)  they ADD UP
w.grad.zero_()               # reset (in place)
with torch.no_grad():        # nothing recorded here
    w -= 0.1 * 18            # a manual update
print(w)                     # tensor(-0.8000, requires_grad=True)
print((2 * w).detach().requires_grad)   # False
```

Gradients accumulate

`backward()` **adds** to `.grad`. In a loop, reset before each backward: `optimizer.zero_grad()`.

- `torch.no_grad()`: the update and the evaluation are not part of the model, do not record them.
- `t.detach()`: same values, cut from the graph (to store or plot).

nn.Module: a model is a class

```python
import torch.nn as nn
class MLP(nn.Module):
    def __init__(self):
        super().__init__()                # always first
        self.hidden = nn.Linear(64, 32)  # W1 (32, 64), b1 (32,)
        self.out = nn.Linear(32, 10)     # W2 (10, 32), b2 (10,)
    def forward(self, x):                 # x: (batch, 64)
        a1 = torch.sigmoid(self.hidden(x))
        return self.out(a1)               # logits (batch, 10)
model = MLP()
print(sum(p.numel() for p in model.parameters()))  # 2410
print(model(torch.randn(5, 64)).shape)   # torch.Size([5, 10])
```

- `__init__` **declares** the layers; stored as attributes, their weights are registered in `model.parameters()`.
- `forward` **uses** them. Call `model(x)`, never `model.forward(x)`.
- `nn.Linear(n_in, n_out)` computes `x @ W.T + b`, i.e. $Z = WA + b$ with examples in rows.

$$\underbrace{64 \times 32 + 32}_{\text{layer 1}} + \underbrace{32 \times 10 + 10}_{\text{layer 2}} = 2410$$

Dataset and DataLoader

```
from torch.utils.data import Dataset, DataLoader
class DigitsDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y          # (N, 64), (N,)
    def __len__(self):
        return len(self.y)             # how many examples
    def __getitem__(self, i):
        return self.X[i], self.y[i]    # example number i
ds = DigitsDataset(torch.randn(100, 64), torch.randint(0, 10, (100,)))
loader = DataLoader(ds, batch_size=32, shuffle=True)
xb, yb = next(iter(loader))
print(len(ds), len(loader), xb.shape, yb.shape)
# 100 4 torch.Size([32, 64]) torch.Size([32])
```

- `Dataset` : how to read **one** example (from memory here, from disk for real images).
- `DataLoader` : groups examples into **batches**, stacks them into tensors.
- `shuffle=True` for training (new order each epoch), `False` for validation and test.
- 100 examples, batches of 32: 32, 32, 32, 4, so 4 batches.

Mini-batch gradient descent

In Sessions 2 and 3, one gradient step used all m examples. Now one step uses one batch of B examples:

$$\nabla_\theta J \;=\; \frac{1}{m}\sum_{i=1}^{m}\nabla_\theta \mathcal{L}^{(i)} \;\approx\; \frac{1}{B}\sum_{i\in\text{batch}}\nabla_\theta \mathcal{L}^{(i)}, \qquad \theta \leftarrow \theta - \eta\,\nabla_\theta J$$

- The batch gradient is a **noisy estimate** of the full gradient, but m/B times cheaper.
- Many cheap, slightly noisy steps beat a few exact ones, and the data does not need to fit in memory.

Vocabulary

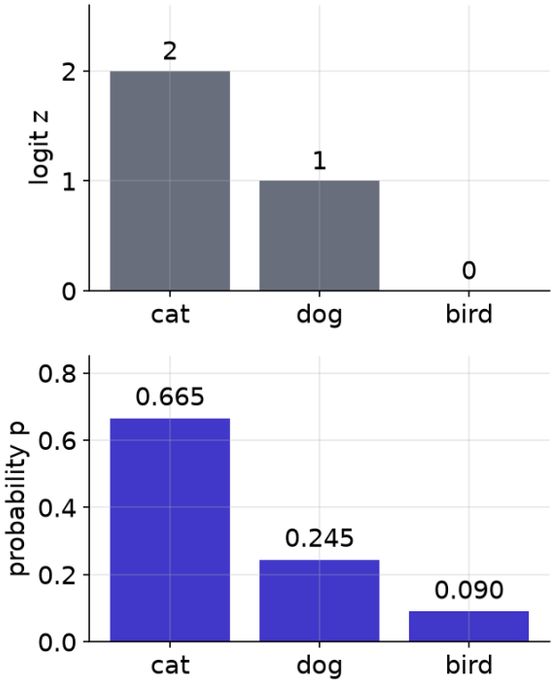
Step (iteration): one batch, one update. **Epoch**: one pass over the training set, $\lceil m/B \rceil$ steps. Digits: 1197 training images, $B = 32$: 38 steps per epoch.

From 2 classes to C classes: the softmax

The output layer gives C raw scores, the **logits** $z = (z_1, \dots, z_C)$. The softmax turns them into probabilities:

$$p_k = \mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}$$

- $p_k > 0$ and $\sum_k p_k = 1$: a probability distribution.
- The exponential keeps the order: largest logit, largest probability. Prediction: $\arg\max_k z_k$.
- It amplifies the gaps: one logit 1 above the others means $e \approx 2.7$ times more likely.



Logits $(2, 1, 0)$ and their softmax.

Two properties of the softmax

1. Adding a constant changes nothing

$$\frac{e^{z_k+c}}{\sum_j e^{z_j+c}} = \frac{e^c\, e^{z_k}}{e^c \sum_j e^{z_j}} = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Only the **differences** between logits matter. Libraries subtract $\max_j z_j$ before the exponential so that e^z never overflows.

2. With $C = 2$ it is the sigmoid

$$p_1 = \frac{e^{z_1}}{e^{z_0} + e^{z_1}} = \frac{1}{1 + e^{-(z_1 - z_0)}} = \sigma(z_1 - z_0)$$

Divide numerator and denominator by e^{z_1}. The softmax is the natural extension of the sigmoid neuron of Session 2.

Exercise 1 · A softmax by hand

A network outputs the logits $z = (2, 1, 0)$ for the classes (cat, dog, bird). Use $e^2 \approx 7.389$ and $e^1 \approx 2.718$.

Exercise 8 min pen and paper

1. Compute the three probabilities p_1, p_2, p_3. Check that they sum to 1.
2. Which class is predicted? How much more likely is it than the second one?
3. The logits become $(12, 11, 10)$. What are the new probabilities? Why?
4. The logits become $(4, 2, 0)$ (twice the original). Is the network more or less confident?

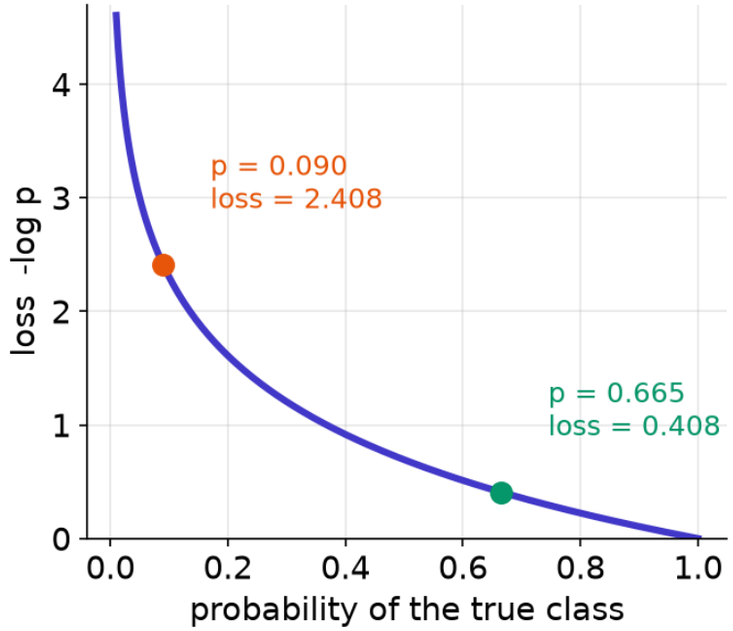
Your notes / answers (saved in this browser)

The cross-entropy loss

Encode the label as a one-hot vector: class 2 of 3 is $y = (0, 1, 0)$. The loss of one example:

$$\mathcal{L} = -\sum_{k=1}^{C} y_k \log p_k = -\log p_{\text{true}}$$

- Only the probability of the true class counts: the sum keeps one term.
- $p_{\text{true}} = 1$: loss 0. $p_{\text{true}} \to 0$: loss $\to \infty$. Confident mistakes cost a lot.
- It is the negative log-likelihood, as in Session 2. With $C = 2$ it is exactly the binary log loss $-[y \log a + (1 - y) \log(1 - a)]$.
- Cost over a batch: the mean $J = \frac{1}{B} \sum_i \mathcal{L}^{(i)}$.



$-\log p_{\text{true}}$: 0 when right and sure, huge when wrong and sure.

The gradient of softmax + cross-entropy

Write the loss with the logits. With c the true class, $\log p_c = z_c - \log \sum_k e^{z_k}$, so

$$\mathcal{L} = -z_c + \log \sum_{k=1}^{C} e^{z_k}$$

Differentiate with respect to one logit z_j:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \underbrace{-1[j = c]}_{-y_j} + \underbrace{\frac{e^{z_j}}{\sum_k e^{z_k}}}_{p_j} = p_j - y_j \qquad \Longrightarrow \qquad \frac{\partial \mathcal{L}}{\partial z} = p - y$$

Same form as Session 2

Prediction minus truth, $a - y$ for the sigmoid, $p - y$ for the softmax. Backpropagation then continues exactly as in Session 3, with $dZ^{[L]} = P - Y$.

Exercise 2 · Cross-entropy by hand

Same network as Exercise 1: $p = (0.665, 0.245, 0.090)$. Use $\ln 0.665 \approx -0.408$, $\ln 0.090 \approx -2.408$, $\ln 10 \approx 2.303$.

Exercise 8 min pen and paper

1. The true class is cat (class 1). Compute the loss $\mathcal{L}$.
2. The true class is bird (class 3). Compute $\mathcal{L}$. Compare.
3. For the true class cat, give the gradient $\partial \mathcal{L} / \partial z$. Which logits will gradient descent push up, which down?
4. An untrained network on 10 classes outputs roughly $p_k = 1/10$ for every class. What loss do you expect at the start of training?

Your notes / answers (saved in this browser)

nn.CrossEntropyLoss in PyTorch

```
loss_fn = nn.CrossEntropyLoss()
logits = torch.tensor([[2., 1., 0.],      # 2 examples
                       [2., 1., 0.]],     # raw scores
                      requires_grad=True)
y = torch.tensor([0, 2])                  # int64 labels
loss = loss_fn(logits, y)                 # mean over batch
loss.backward()
print(round(loss.item(), 4))   # 1.4076 = (0.408 + 2.408) / 2
print(logits.grad)             # (p - y) / 2 for each row
# tensor([[-0.1674,  0.1224,  0.0450],
#         [ 0.3326,  0.1224, -0.4550]])
```

Two classic mistakes

- **No softmax before the loss**: it applies log-softmax itself. Give it the **logits**.
- **Labels**: class indices of shape `(B,)` and dtype `int64`, not one-hot, not float.

Classes are numbered from 0 in code. The gradient is $(p - y)/B$ because the loss is a mean over the $B = 2$ examples.

The optimiser applies the update rule

```
model = nn.Linear(2, 1)
opt = torch.optim.SGD(model.parameters(), lr=0.1)
x, y = torch.tensor([[1., 2.]]), torch.tensor([[1.]])
loss = ((model(x) - y) ** 2).mean()
opt.zero_grad()                    # p.grad = 0 for every p
loss.backward()                    # fills p.grad
before = model.weight.detach().clone()
opt.step()                         # p ← p - lr * p.grad
print(torch.allclose(model.weight,
      before - 0.1 * model.weight.grad))   # True
```

- `SGD(model.parameters(), lr)` receives the list of tensors to update.
- `step()` does, for every parameter, what we wrote by hand: `with torch.no_grad(): p -= lr * p.grad`.
- It does **not** compute the gradients: `backward()` does.

> **Other optimisers**
>
> `optim.Adam(params, lr=1e-3)` adapts the step of each parameter. Same two methods. Details in Session 6.

The training loop, line by line

```
for epoch in range(n_epochs):                # one epoch = one pass over the training set
    model.train()                            # training mode (matters for dropout, batch norm)
    for xb, yb in train_loader:              # one mini-batch at a time
        xb, yb = xb.to(device), yb.to(device)   # data on the same device as the model
        logits = model(xb)                   # 1. forward pass: (B, 10) logits
        loss = loss_fn(logits, yb)           # 2. loss: mean cross-entropy on the batch
        optimizer.zero_grad()                # 3. reset the gradients of the previous step
        loss.backward()                      # 4. backpropagation: fills p.grad for every p
        optimizer.step()                     # 5. update: p ← p - lr * p.grad
    val_loss, val_acc = evaluate(model, val_loader, loss_fn)   # after each epoch
```

The five lines

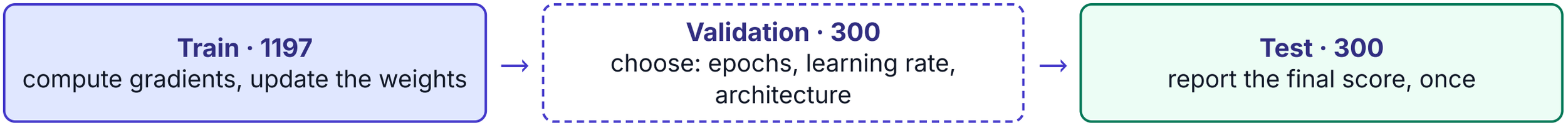
Forward, loss, zero_grad, backward, step. Learn them by heart: every PyTorch training, from this MLP to a large language model, has them.

The evaluation loop

```python
def evaluate(model, loader, loss_fn):
    model.eval()                            # evaluation mode
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():                   # no graph needed
        for xb, yb in loader:
            logits = model(xb)
            total_loss += loss_fn(logits, yb).item() * len(yb)
            pred = logits.argmax(dim=1)  # largest logit
            correct += (pred == yb).sum().item()
            total += len(yb)
    return total_loss / total, correct / total
```

- `model.eval()`: some layers (dropout, batch norm, Session 6) behave differently at test time. `model.train()` switches back.
- `torch.no_grad()`: faster, less memory.
- `argmax(dim=1)`: the predicted class of each row; no softmax needed.
- `* len(yb)`: the last batch can be smaller, so we weight each batch mean by its size.

Train, validation, test



- The training score is optimistic: the model has seen these examples.
- If you pick the best of 20 settings on a set, that set's score is optimistic too: you selected on it. So we choose on **validation**.
- The **test** set is touched once, at the end, with the chosen model: an honest estimate on new data.
- Split with a fixed seed (`torch.randperm` with a generator) so that the split is reproducible.

Saving a model, and devices

```
torch.save(model.state_dict(), "digits_mlp.pt")
print(list(model.state_dict().keys()))
# ['hidden.weight', 'hidden.bias', 'out.weight', 'out.bias']
model2 = MLP()                       # same class, random weights
model2.load_state_dict(torch.load("digits_mlp.pt"))
model2.eval()                        # ready to predict
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = MLP().to(device)       # moves the parameters
xb, yb = xb.to(device), yb.to(device)   # and every batch
```

`state_dict()`: a dictionary name to tensor, the learned numbers only. The code of the class is needed to rebuild the model.

On Colab: Runtime > Change runtime type > GPU. Model and data must be on the same device, otherwise: `RuntimeError: Expected all tensors to be on the same device`.

Exercise 3 · Find the five bugs

```
model = MLP()
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
for epoch in range(30):
    for xb, yb in train_loader:
        probs = torch.softmax(model(xb), dim=1)
        loss = loss_fn(probs, yb.float())
        loss.backward()
        optimizer.step()
correct = 0
with torch.no_grad():
    for xb, yb in train_loader:
        correct += (model(xb).argmax(dim=1) == yb).sum().item()
print("accuracy:", correct / len(train_loader.dataset))
```

Exercise 8 min read the code

1. Find five bugs (or bad habits). Give the line number of each.
2. For each one, say what goes wrong and write the fix.

Hint: two are in the loss, one is a missing line, two are in the evaluation.

Your notes / answers (saved in this browser)

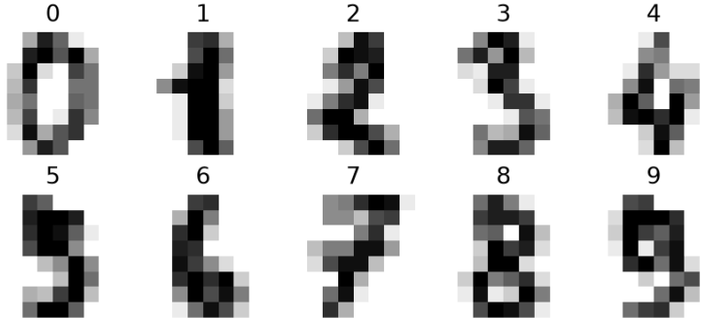
TP · Your PyTorch reference notebook (1)

TP 30 min tp/TP04_pytorch_in_depth.ipynb, Parts A to C

1. Part A: tensors (creation, indexing, operations, broadcasting, NumPy).
2. Part B: autograd (a derivative, the sigmoid neuron of S02, accumulation, gradient descent by hand).
3. Part C: the digits as tensors, train / validation / test, your `Dataset`, the `DataLoader`.

Deliverable: Exercises 1 to 14 print "Exercise validated!".

Your notes / answers (saved in this browser)



scikit-learn digits: 8×8 pixels, values 0 to 16.

Break: 10 minutes

TP · Your PyTorch reference notebook (2)

TP 55 min tp/TP04_pytorch_in_depth.ipynb, Parts D to F

1. Part D: the 64-32-10 sigmoid network as an `nn.Module`, its 2410 parameters, logits to probabilities.
2. Part E: `CrossEntropyLoss` (check the initial loss is about 2.30), one SGD step, `evaluate`, then the full training loop, 30 epochs.
3. Part F: save, reload, identical predictions, the test accuracy measured once.
4. Going further: `train_one_epoch` and `fit` (reused in Sessions 5 and 6), Adam in one line.

Deliverable: validation accuracy above 0.9 after 30 epochs; a test accuracy you report once.

Your notes / answers (saved in this browser)

Watch it on YouTube Machine Learnia, Deep Learning series

What PyTorch does for you, coded by hand in NumPy.

Video 9

Programmer le réseau à 2 couches avec NumPy

Coding the 2-layer network in NumPy

Video 10

Réseau de neurones à N couches

A neural network with N layers

Playlist

The whole Deep Learning series

10 videos, from the perceptron to the network with N layers

Videos in French. Click a line in your copy of the course to open the video.

Summary

1. Tensor (shape, dtype, device), autograd (`backward` adds into `.grad`), `nn.Module` (declare in `__init__`, use in `forward`).
2. Softmax turns C logits into probabilities; cross-entropy is $-\log p_{\text{true}}$, its gradient $p - y$.
3. `nn.CrossEntropyLoss` takes raw logits and int64 labels.
4. The five lines: forward, loss, `zero_grad`, `backward`, `step`. Evaluate with `eval()` and `no_grad()`.
5. Choose on validation, report on test once; save the `state_dict`.

Coding a full neural network from scratch in NumPy (forward and backward, layer by layer) is a great exercise: I do it step by step on my YouTube channel Machine Learnia, in videos 9 and 10 of the Deep Learning series (previous slide).

Project checkpoint: a dense-network baseline on your artworks, trained with this clean training loop (tag `m1`).

Next session: a computer-vision project with this network, its limits when it gets deeper, and the fixes (ReLU, initialisation).